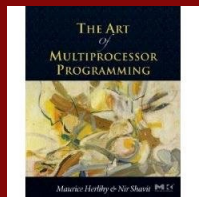
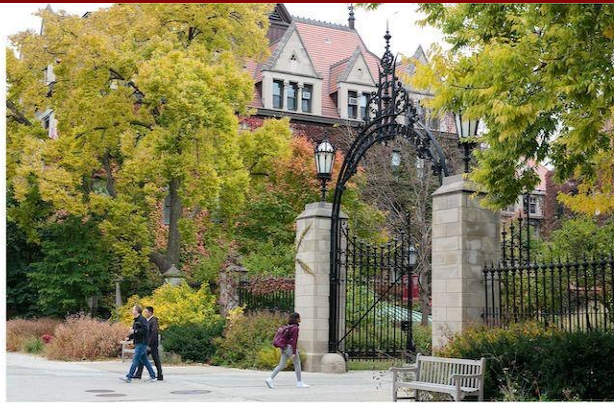


MPCS 52060 - Parallel Programming

M7: Advanced Parallel Patterns and Techniques (Part 1)

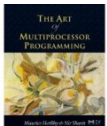
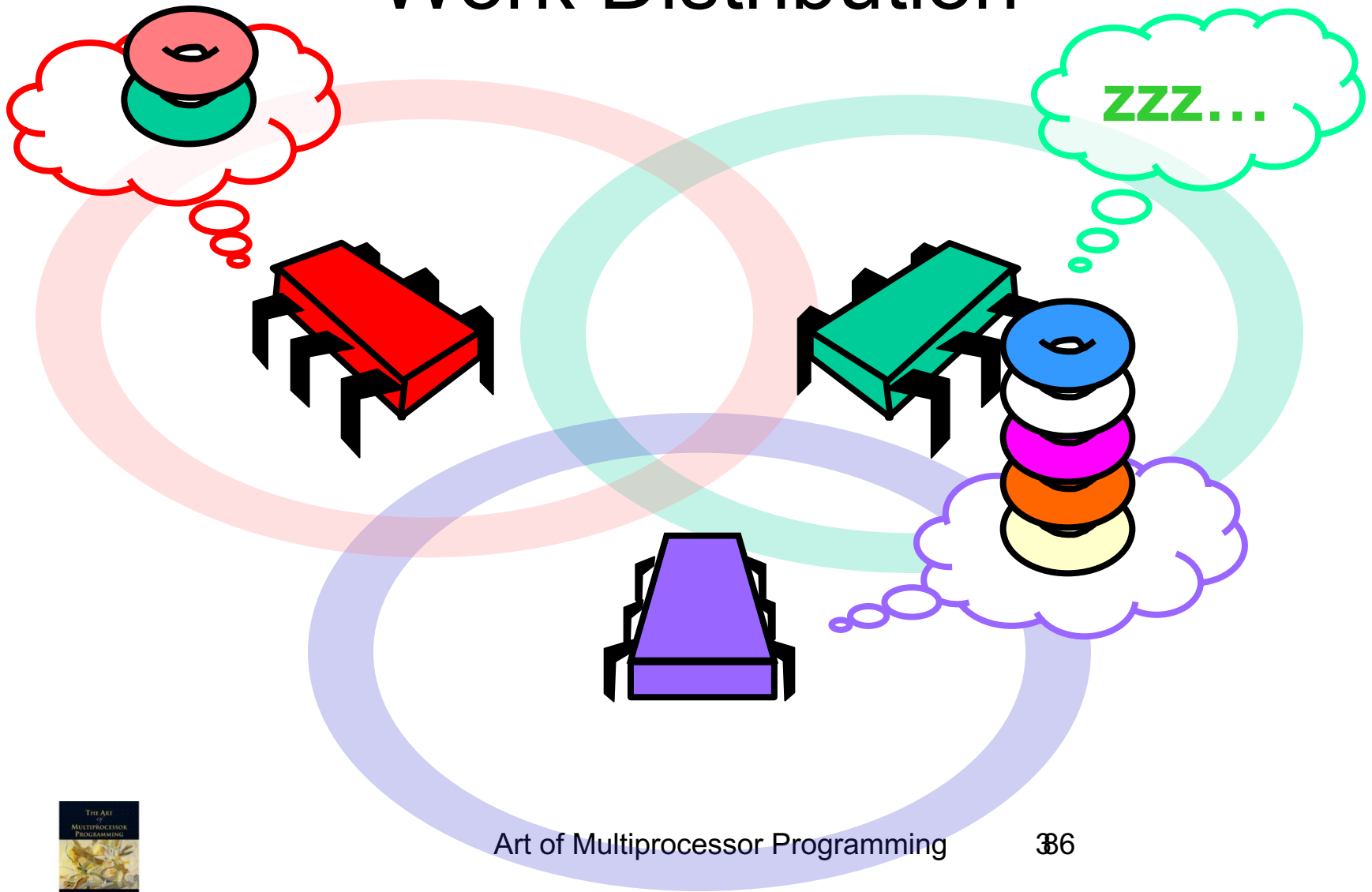


Original slides from “The Art of Multiprocessor Programming by Maurice Herlihy & Nir Shavit” With modifications by Lamont Samuels and Jan Hückelheim

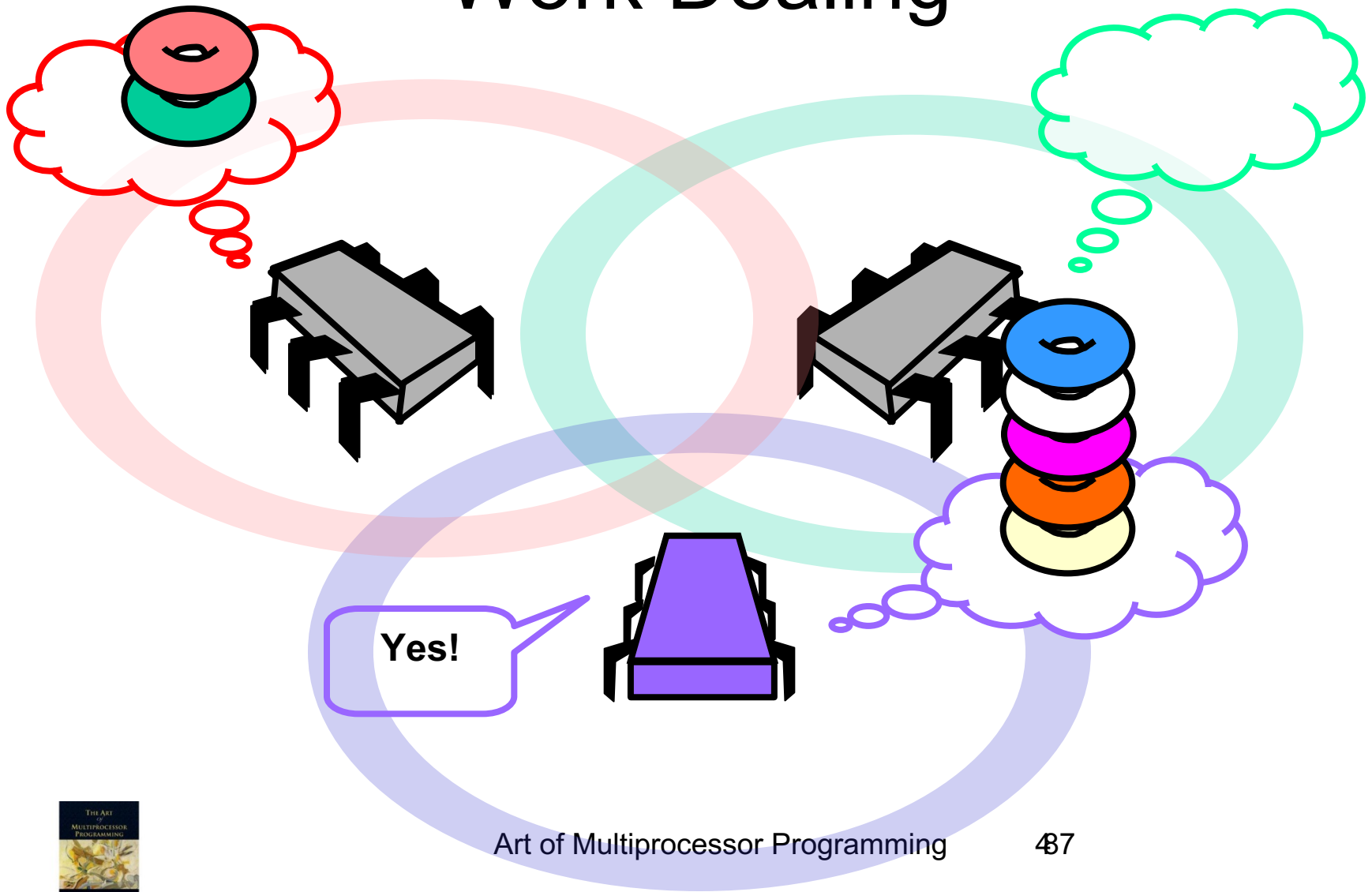
Recap and Outlook

- Last week:
 - Channels
 - Go thread / coroutine scheduling
- This week:
 - Deeper look into scheduling
 - Programming patterns

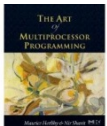
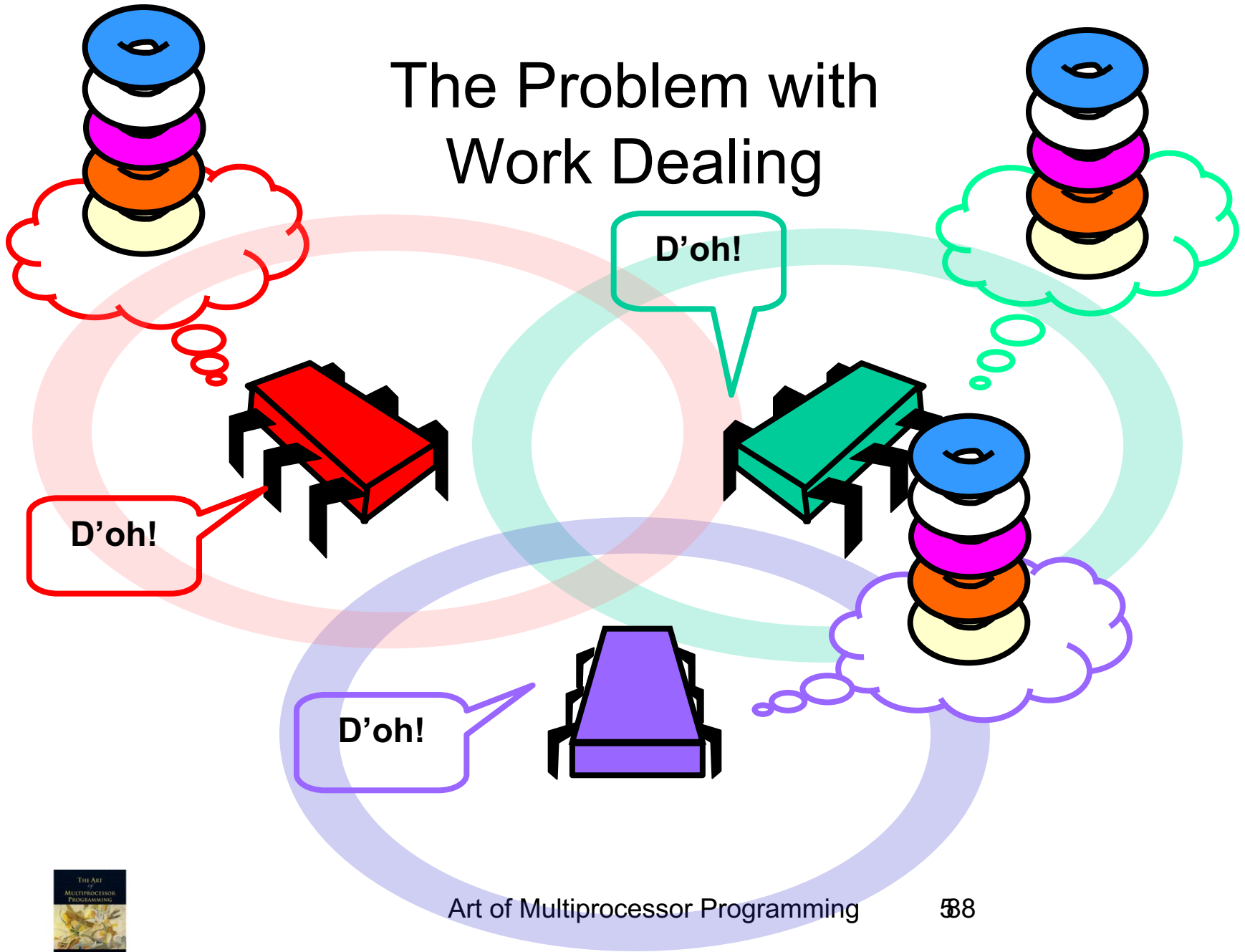
Work Distribution



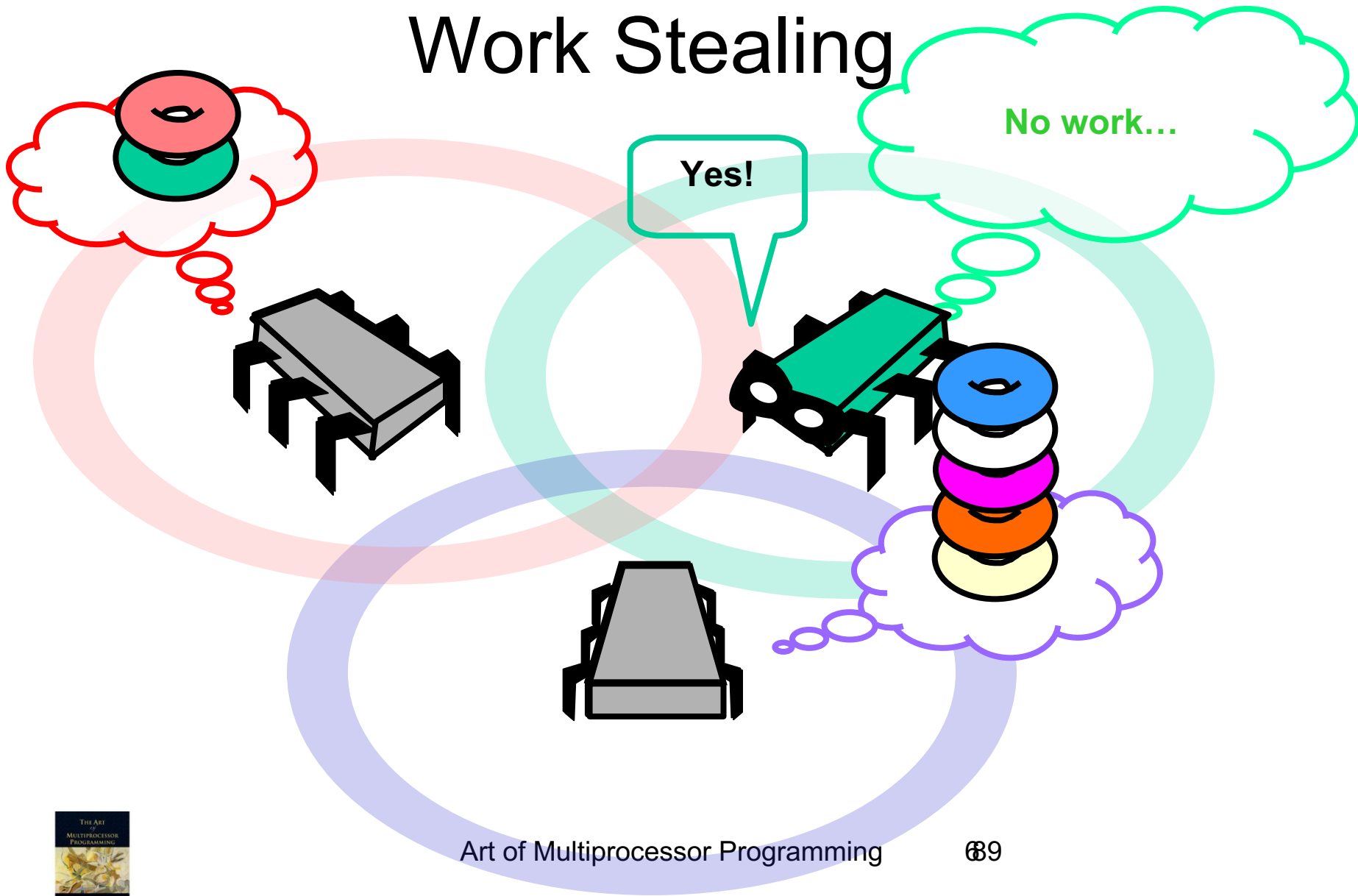
Work Dealing



The Problem with Work Dealing

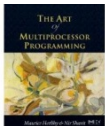


Work Stealing



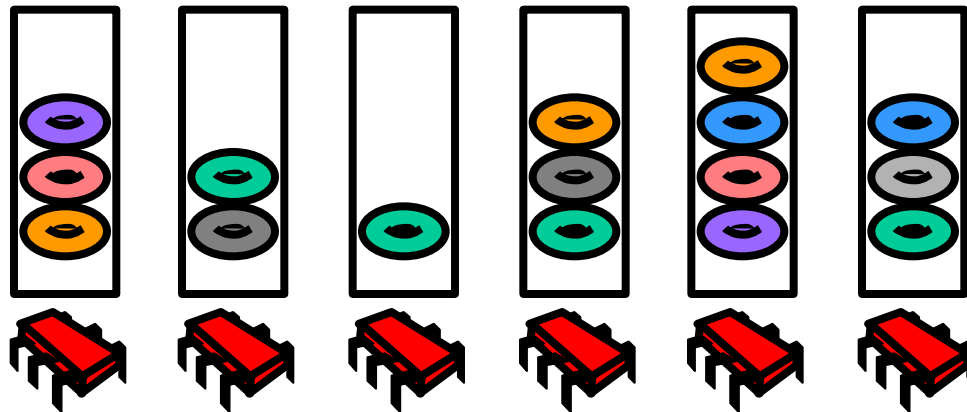
Lock-Free Work Stealing

- Each thread has a pool of ready work
- Remove work without synchronizing
- If you run out of work, steal someone else's
- Choose victim at random



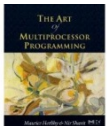
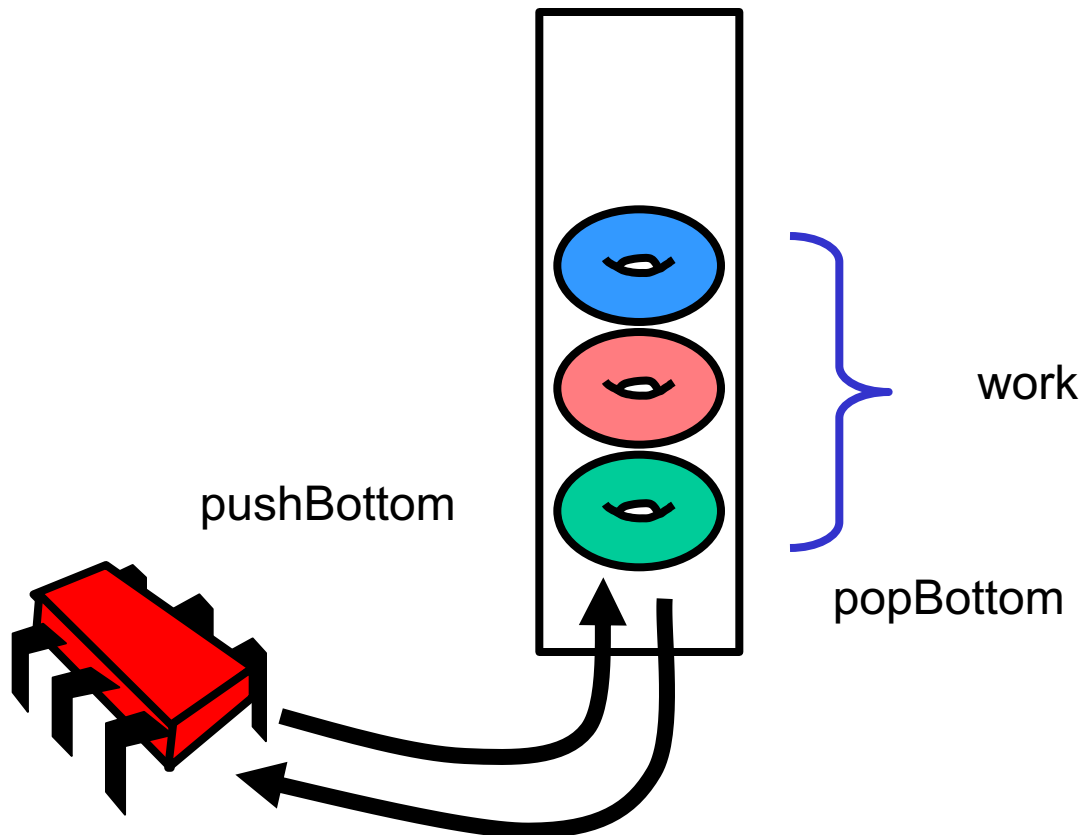
Local Work Pools

Each work pool is a Double-Ended Queue



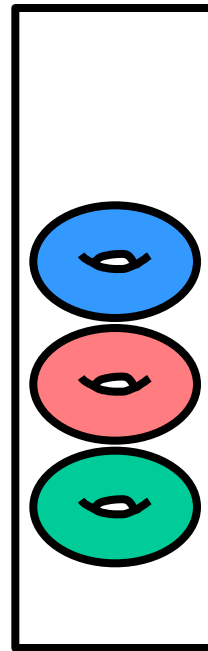
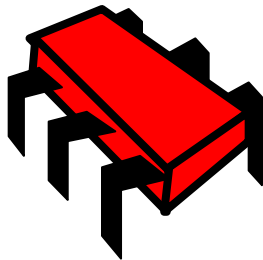
Work DEQueue¹

1. Double-Ended Queue

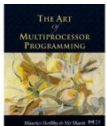


Obtain Work

- Obtain work
- Run task until
- Blocks or terminates

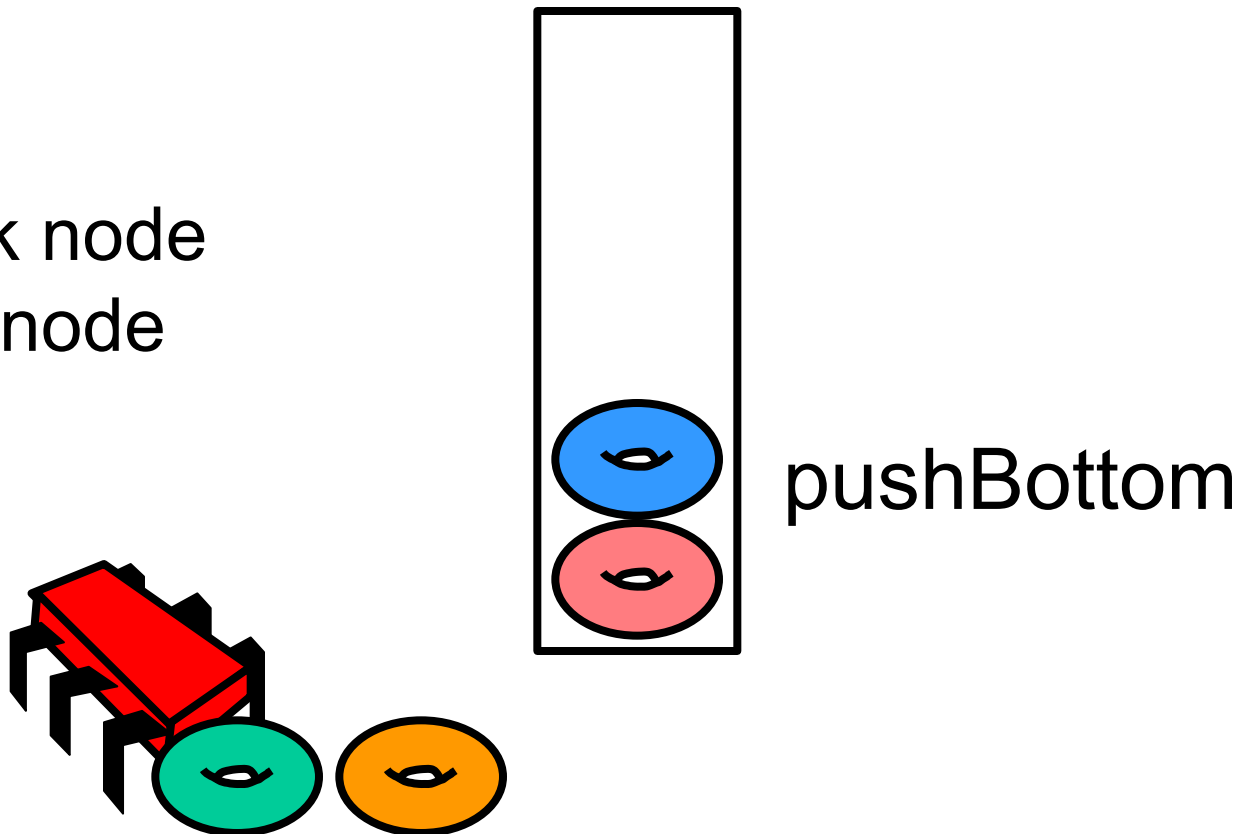


popBottom

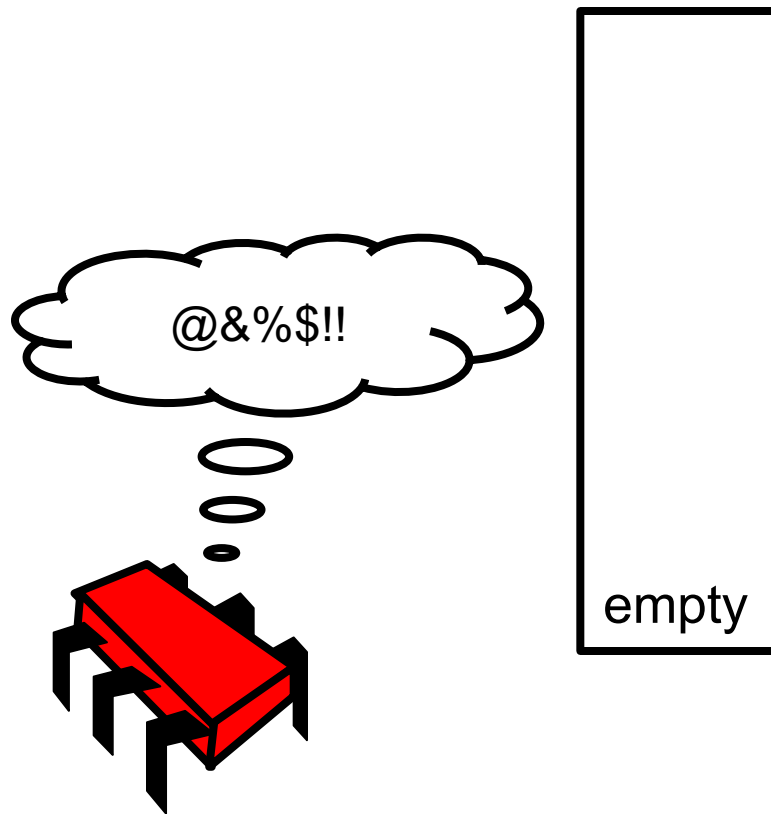


New Work

- Unblock node
- Spawn node

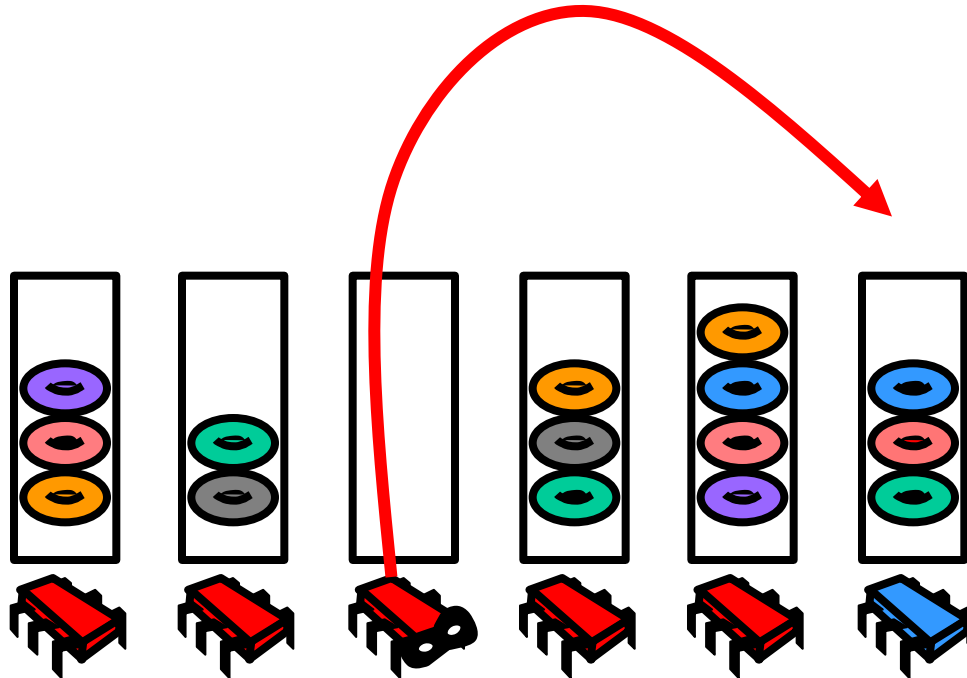


Whatcha Gonna do When the Well Runs Dry?

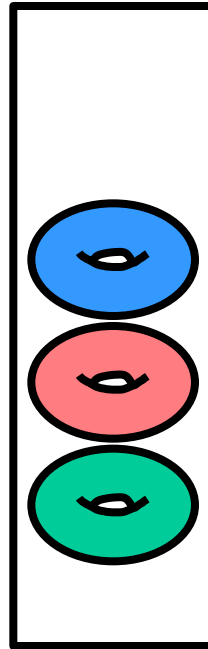
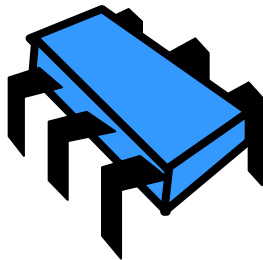


Steal Work from Others

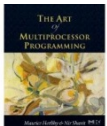
Pick random thread's DEQueue



Steal this Task!

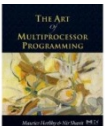


popTop



Task DEQueue

- Methods
 - pushBottom
 - popBottom
 - popTop
- } Never happen concurrently



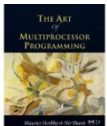
Task DEQueue

- Methods

- pushBottom
- popBottom
- popTop



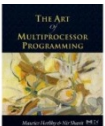
Most common –
make them fast
(minimize use of
CAS)



Ideal

- Wait-Free
- Linearizable
- Constant time

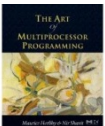
What is linearizable?



Ideal

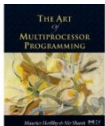
- Wait-Free
- Linearizable
- Constant time

We don't know how to get all three!

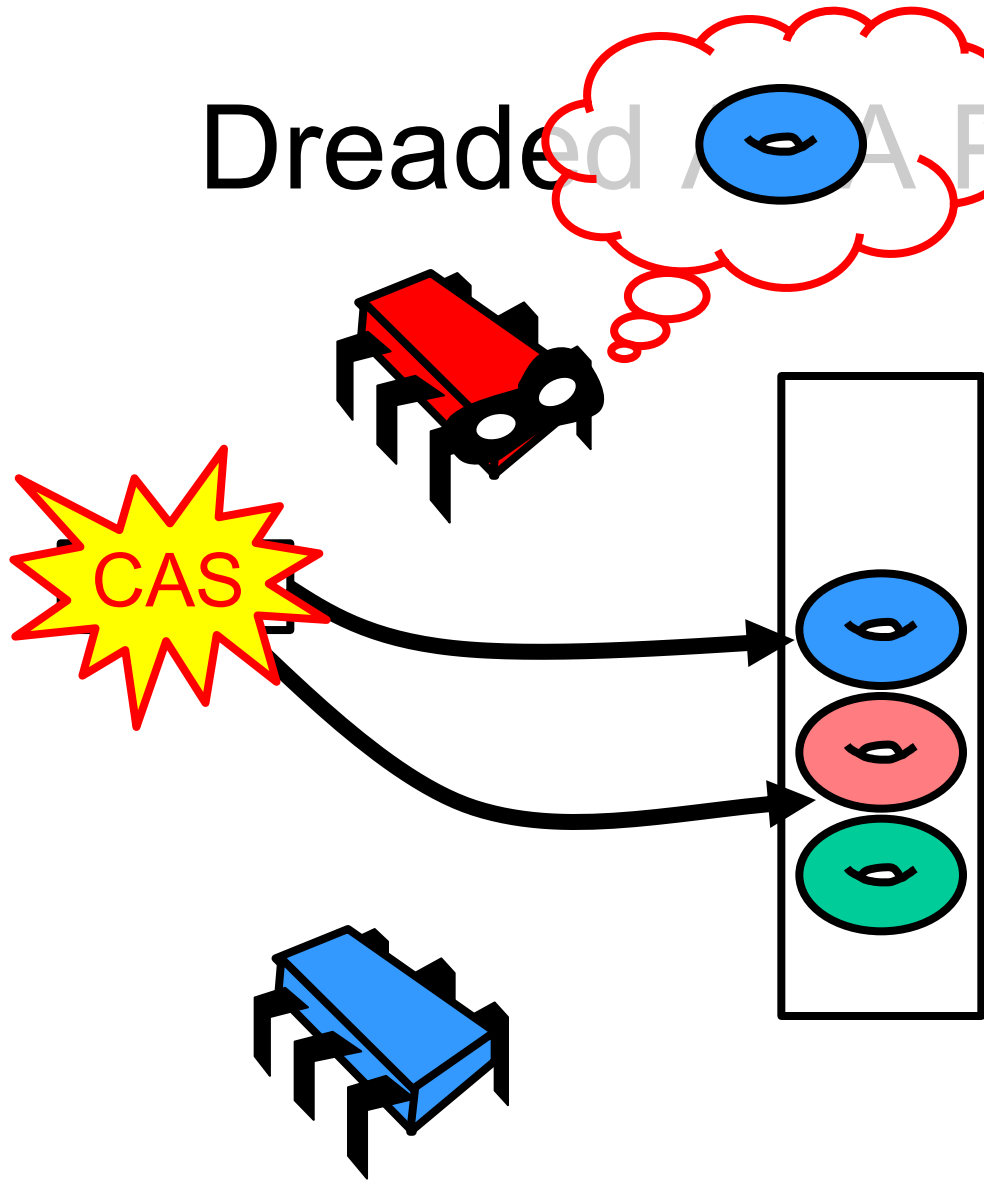


Compromise

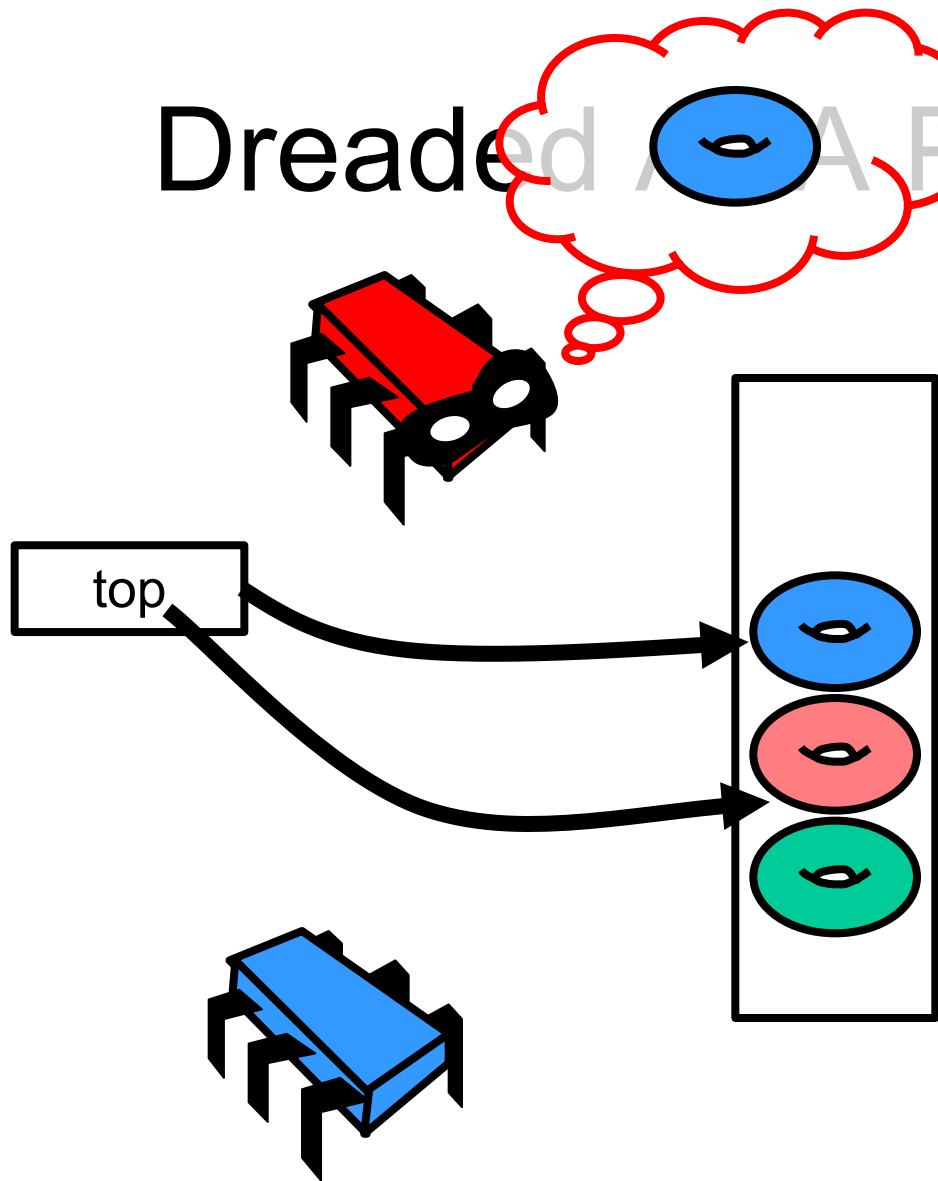
- Method popTop may fail if
 - Concurrent popTop succeeds, or a
 - Concurrent popBottom takes last task



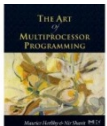
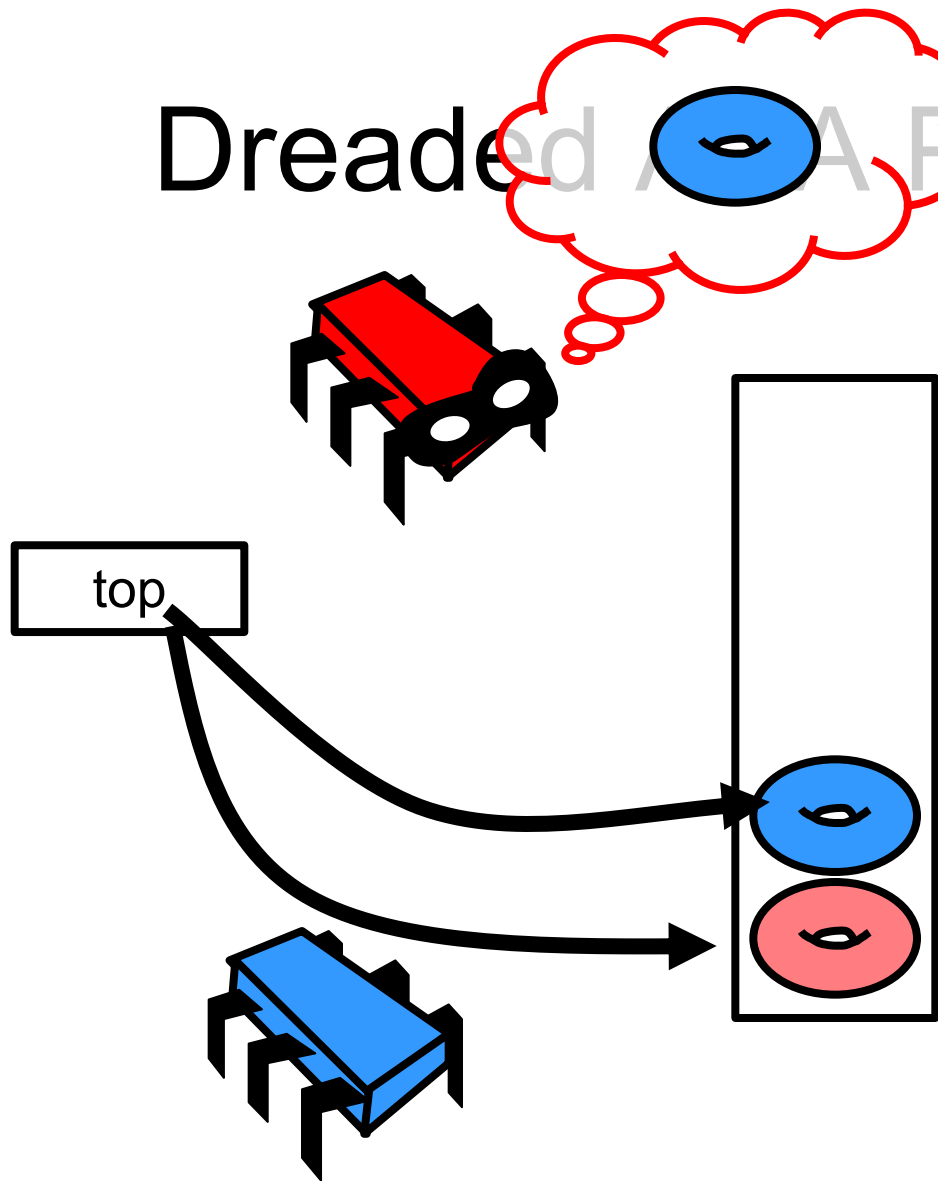
Dreaded A Problem



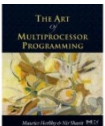
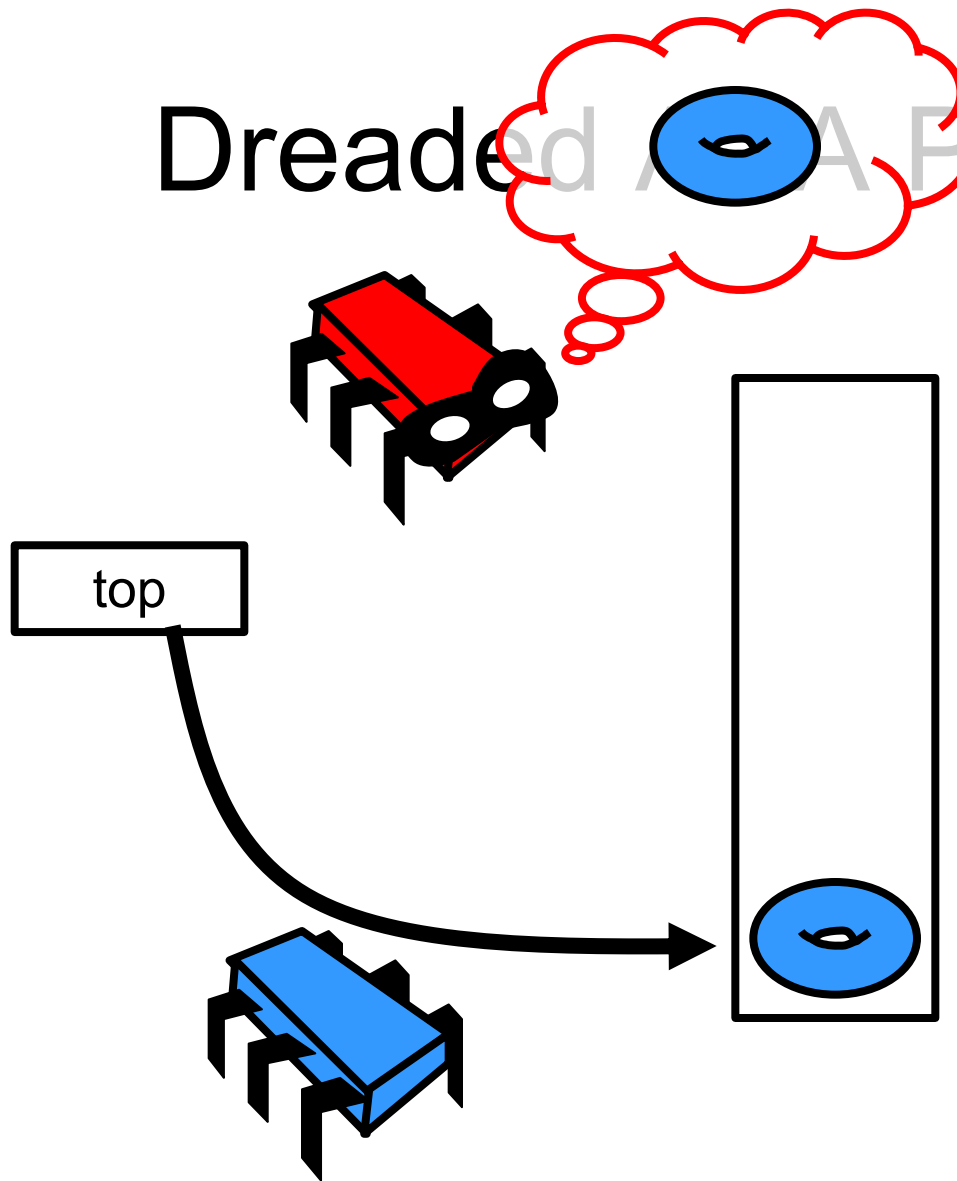
Dreaded A Problem



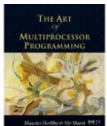
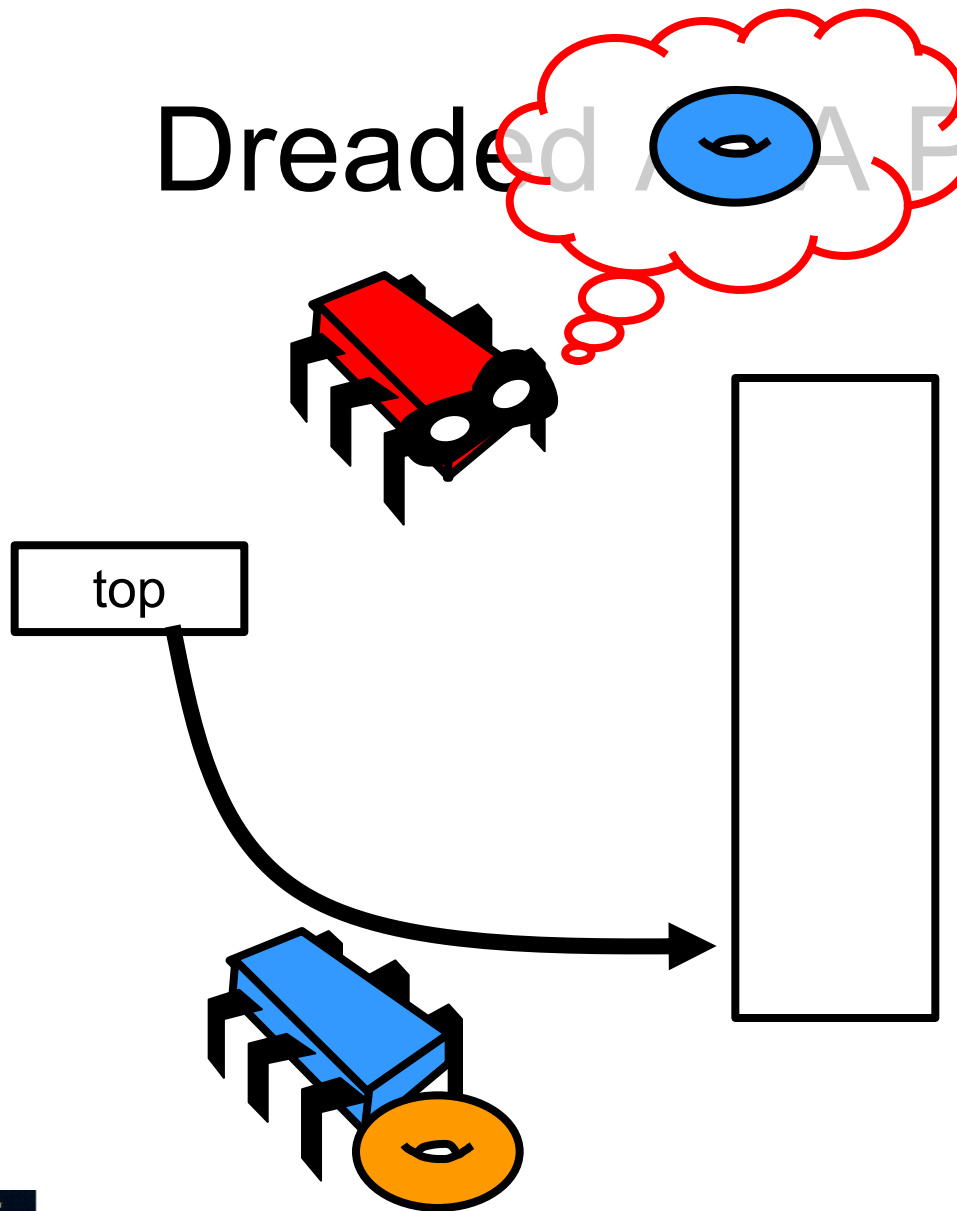
Dreaded A Problem



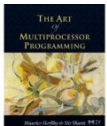
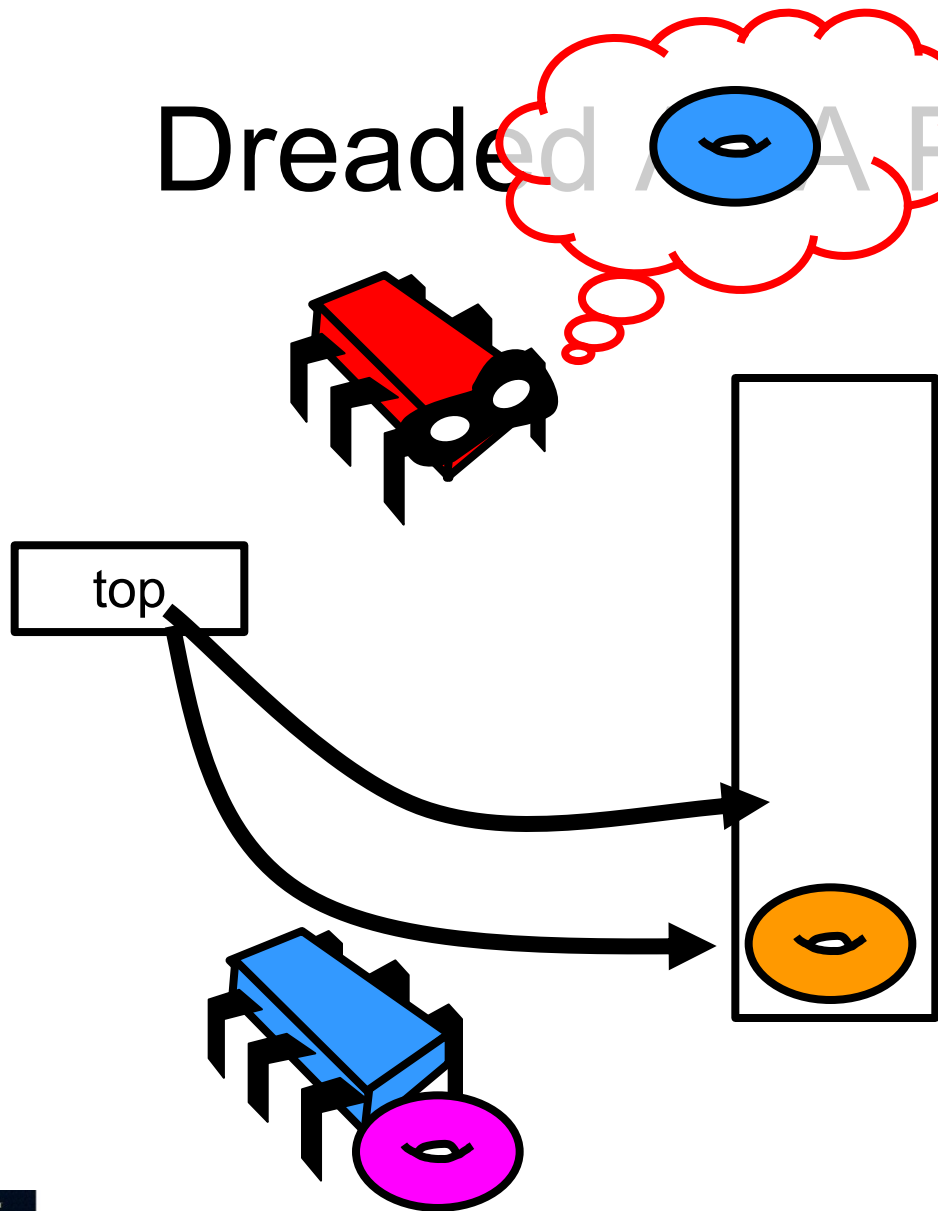
Dreaded A Problem



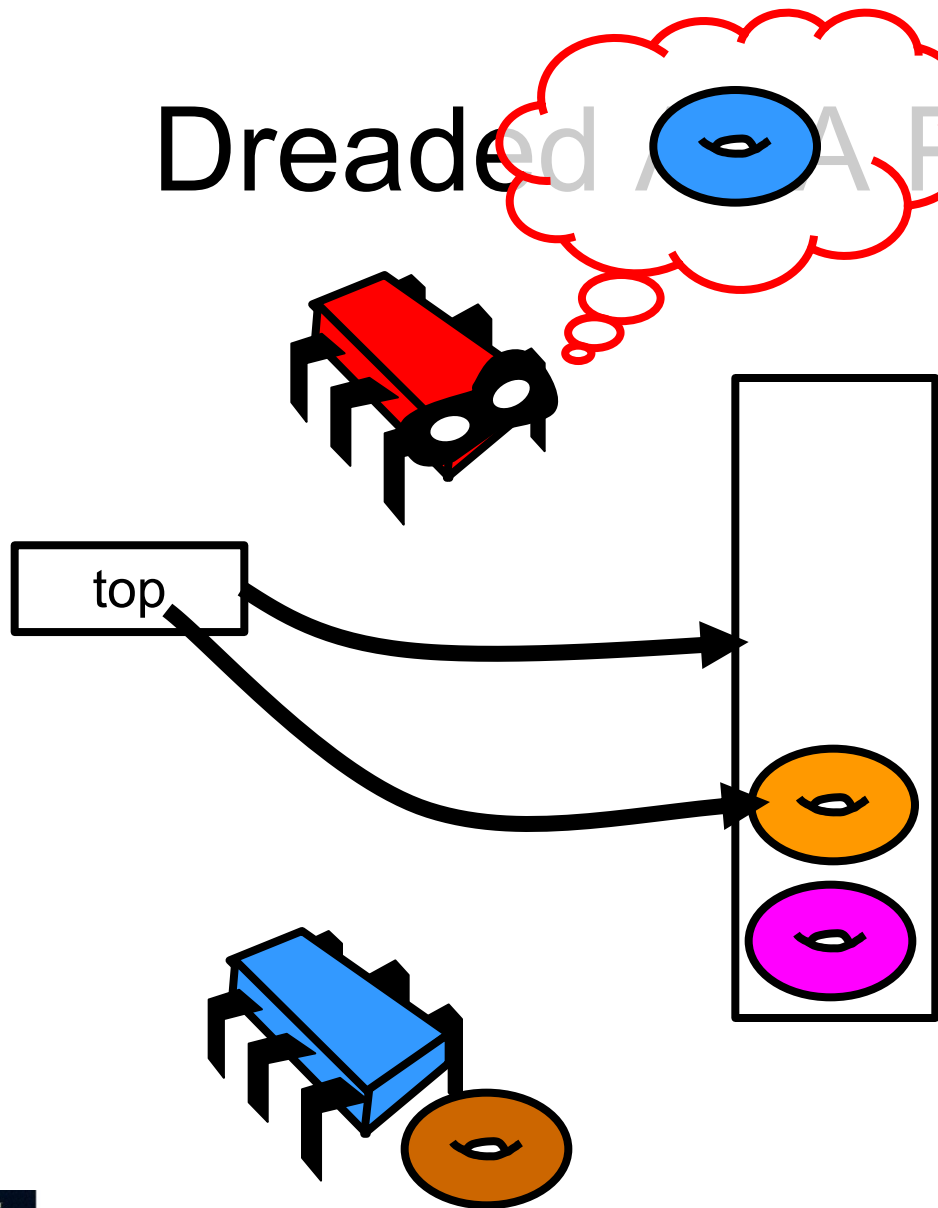
Dreaded A Problem



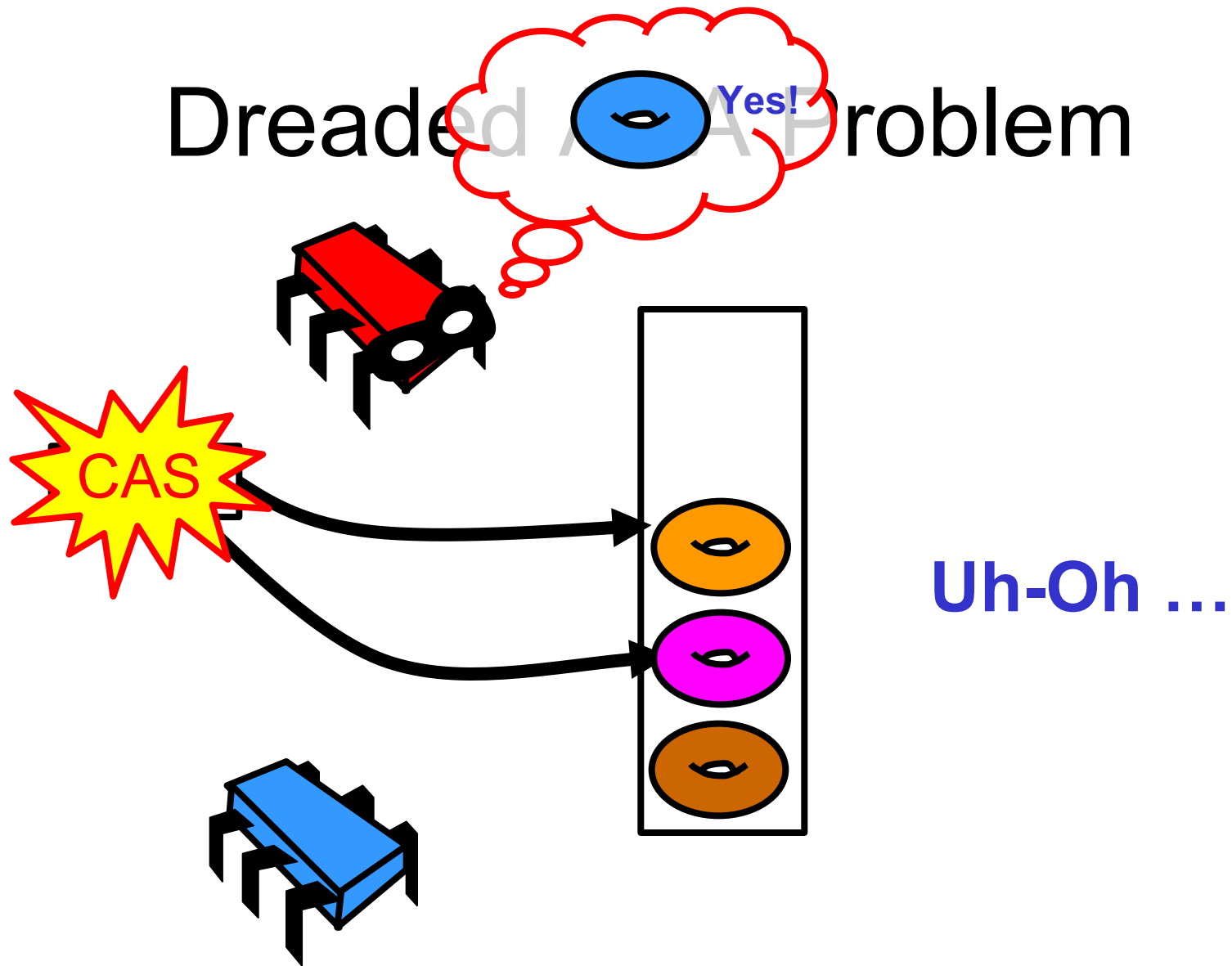
Dreaded A Problem



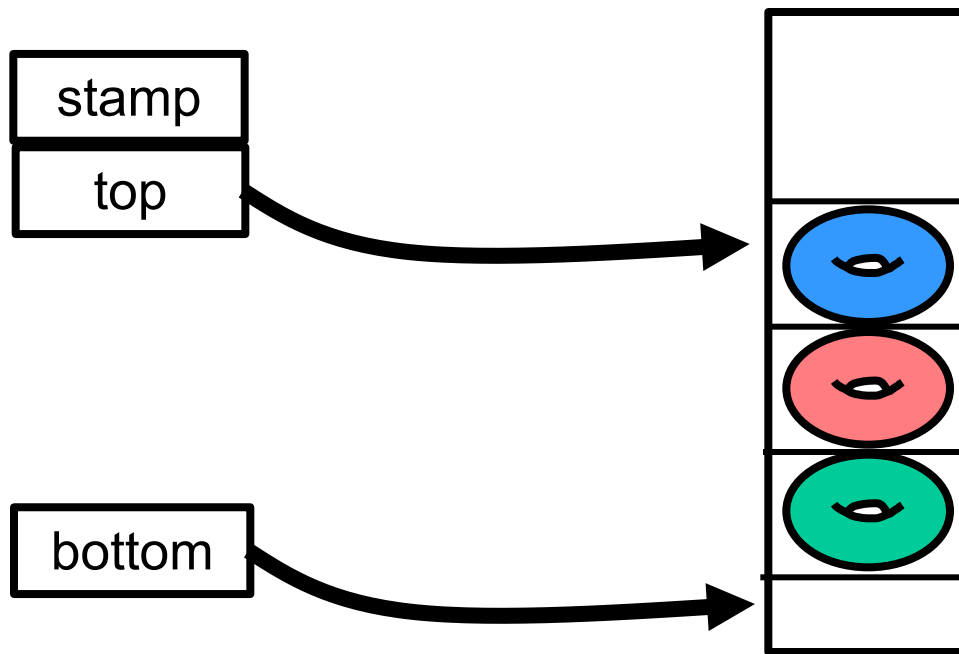
Dreaded A Problem



Dreaded A/P Problem

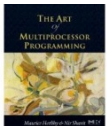


Fix for Dreaded ABA



Bounded DEQueue

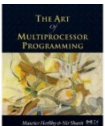
```
public class BDEQueue {  
    AtomicStampedReference<Integer> top;  
    volatile int bottom;  
    Runnable[] tasks;  
    ...  
}
```



Bounded DEQueue

```
public class BDEQueue {  
    AtomicStampedReference<Integer> top;  
    volatile int bottom;  
    Runnable[] tasks;  
    ...  
}
```

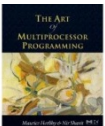
**Index & Stamp
(synchronized)**



Bounded DEQueue

```
public class BDEQueue {  
    AtomicStampedReference<Integer> top;  
    volatile int bottom;  
    Runnable[] deq;  
    ...  
}
```

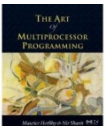
**index of bottom task
no need to synchronize
memory barrier needed**



Bounded DEQueue

```
public class BDEQueue {  
    AtomicStampedReference<Integer> top;  
    volatile int bottom;  
    Runnable[] tasks;  
    ...  
}
```

Array holding tasks



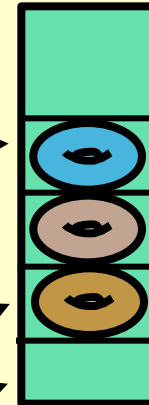
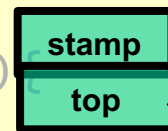
pushBottom()

```
public class BDEQueue {  
    ...  
    void pushBottom(Runnable r) {  
        tasks[bottom] = r;  
        bottom++;  
    }  
    ...  
}
```

**Bottom is the index to store
the new task in the array**

pushBottom()

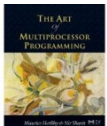
```
public class BDEQueue {  
    ...  
    void pushBottom(Runnable r) {  
        tasks[bottom] = r;  
        bottom++;  
    }  
    ...  
}
```



Adjust the bottom index

Steal Work

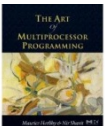
```
public Runnable popTop() {  
    int[] stamp = new int[1];  
    int oldTop = top.get(stamp), newTop = oldTop + 1;  
    int oldStamp = stamp[0], newStamp = oldStamp + 1;  
    if (bottom <= oldTop)  
        return null;  
    Runnable r = tasks[oldTop];  
    if (top.CAS(oldTop, newTop, oldStamp, newStamp))  
        return r;  
    return null;  
}
```



Steal Work

```
public Runnable popTop() {  
    int[] stamp = new int[1];  
    int oldTop = top.get(stamp), newTop = oldTop + 1;  
    int oldStamp = stamp[0], newStamp = oldStamp + 1;  
    if (bottom <= oldTop)  
        return null;  
    Runnable r = tasks[oldTop];  
    if (top.CAS(oldTop, newTop, oldStamp, newStamp))  
        return r;  
    return null;  
}
```

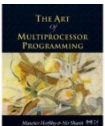
Read top (value & stamp)



Steal Work

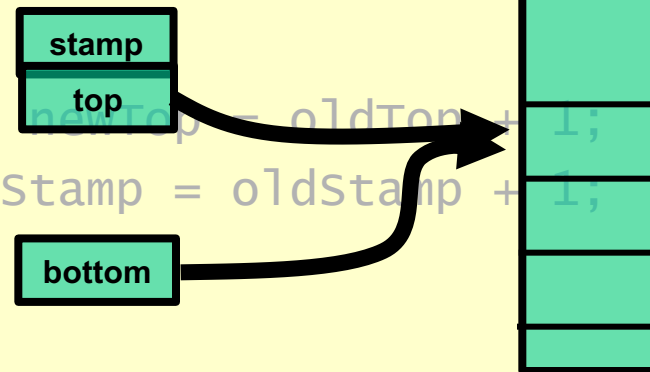
```
public Runnable popTop() {  
    int[] stamp = new int[1];  
    int oldTop = top.get(stamp), newTop = oldTop + 1;  
    int oldStamp = stamp[0], newStamp = oldStamp + 1;  
    if (bottom <= oldTop)  
        return null;  
    Runnable r = tasks[oldTop];  
    if (top.CAS(oldTop, newTop, oldStamp, newStamp))  
        return r;  
    return null;  
}
```

Compute new value & stamp

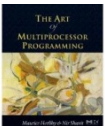


Steal Work

```
public Runnable popTop() {  
    int[] stamp = new int[1];  
    int oldTop = top.get(stamp),  
    int oldStamp = stamp[0], newStamp = oldStamp + 1;  
    if (bottom <= oldTop)  
        return null;  
    Runnable r = tasks[oldTop];  
    if (top.CAS(oldTop, newTop, oldStamp, newStamp))  
        return r;  
    return null;  
}
```

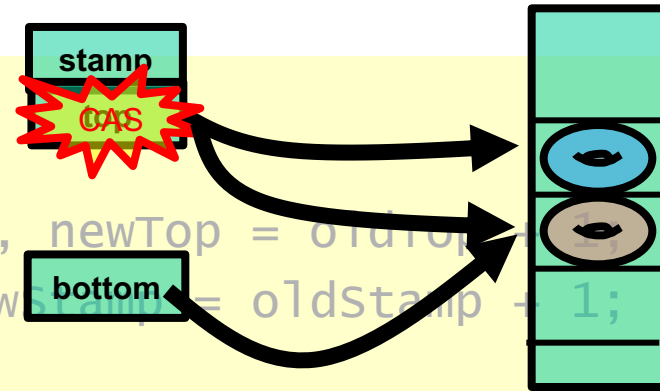


Quit if queue is empty



Steal Work

```
public Runnable popTop() {  
    int[] stamp = new int[1];  
    int oldTop = top.get(stamp), newTop = oldTop - 1;  
    int oldStamp = stamp[0], newStamp = oldStamp + 1;  
    if (bottom <= oldTop)  
        return null;  
    Runnable r = tasks[oldTop];  
    if (top.CAS(oldTop, newTop, oldStamp, newStamp))  
        return r;  
    return null;  
}
```

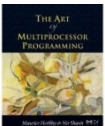


Try to steal the task

Steal Work

```
public Runnable popTop() {  
    int[] stamp = new int[1];  
    int oldTop = top.get(stamp), newTop = oldTop + 1;  
    int oldStamp = stamp[0], newStamp = oldStamp + 1;  
    if (bottom <= oldTop)  
        return null;  
    Runnable r = tasks[oldTop];  
    if (top.CAS(oldTop, newTop, oldStamp, newStamp))  
        return r;  
    return null;  
}
```

**Give up if
conflict occurs**



Take Work

```
Runnable popBottom() {
    if (bottom == 0) return null;
    bottom--;
    Runnable r = tasks[bottom];
    int[] stamp = new int[1];
    int oldTop = top.get(stamp), newTop = 0;
    int oldStamp = stamp[0], newStamp = oldStamp + 1;
    if (bottom > oldTop) return r;
    if (bottom == oldTop){
        bottom = 0;
        if (top.CAS(oldTop, newTop, oldStamp, newStamp))
            return r;
    }
    top.set(newTop, newStamp); bottom = 0;
    return null;}

```



Take Work

```
Runnable popBottom() {  
    if (bottom == 0) return null;  
    bottom--;  
    Runnable r = tasks[bottom];  
    int[] stamp = new int[1];  
    int oldTop = top.get(stamp), newTop = 0;  
    int oldStamp = stamp[0], newStamp = oldStamp + 1;  
    if (bottom > oldTop) return r;  
    if (bottom == oldTop){  
        bottom = 0;  
        if (top.CAS(oldTop, newTop, oldStamp, newStamp))  
            return r;  
    }  
    top.set(newTop, newStamp); bottom = 0;  
    return null;  
}
```

Make sure queue is non-empty

Take Work

```
Runnable popBottom() {  
    if (bottom == 0) return null;  
    bottom--;  
    Runnable r = tasks[bottom];  
    int[] stamp = new int[1];  
    int oldTop = top.get(stamp), newTop = 0;  
    int oldStamp = stamp[0], newStamp = oldStamp + 1;  
    if (bottom > oldTop) return r;  
    if (bottom == oldTop){  
        bottom = 0;  
        if (top.CAS(oldTop, newTop, oldStamp, newStamp))  
            return r;  
    }  
    top.set(newTop, newStamp); bottom = 0;  
    return null;  
}
```

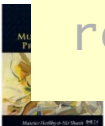
**Prepare to take
bottom task**



Take Work

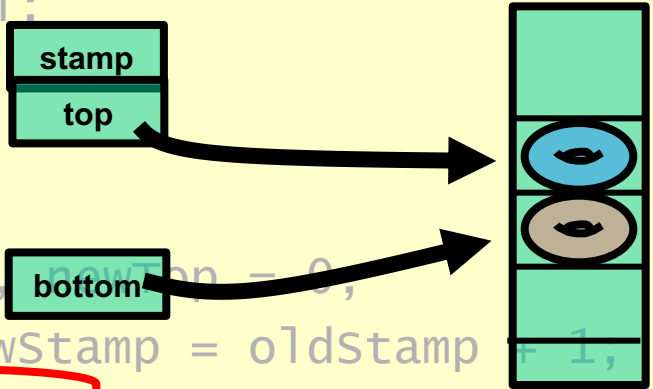
```
Runnable popBottom() {  
    if (bottom == 0) return null;  
    bottom--;  
    Runnable r = tasks[bottom];  
    int[] stamp = new int[1];  
    int oldTop = top.get(stamp), newTop = 0;  
    int oldStamp = stamp[0], newStamp = oldStamp + 1;  
    if (bottom > oldTop) return r;  
    if (bottom == oldTop){  
        bottom = 0;  
        if (top.CAS(oldTop, newTop, oldStamp, newStamp))  
            return r;  
    }  
    top.set(newTop, newStamp); bottom = 0;  
    return null;}  
}
```

**Read top, & prepare
new values**



Take Work

```
Runnable popBottom() {  
    if (bottom == 0) return null;  
    bottom--;  
    Runnable r = tasks[bottom];  
    int[] stamp = new int[1];  
    int oldTop = top.get(stamp), newTop = 0;  
    int oldStamp = stamp[0], newStamp = oldStamp + 1;  
if (bottom > oldTop) return r;  
    if (bottom == oldTop){  
        bottom = 0;  
        if (top.CAS(oldTop, newTop, oldStamp, newStamp))  
            return r;  
    }  
    top.set(newTop, newStamp);  
    return null;  
}
```



**If top & bottom one or more
apart, no conflict**

Take Work

```
Runnable popBottom() {  
    if (bottom == 0) return null;  
    bottom--;  
    Runnable r = tasks[bottom];  
    int[] stamp = new int[1];  
    int oldTop = top.get(stamp), newTop = 0;  
    int oldStamp = stamp[0], newStamp = oldStamp + 1;  
    if (bottom > oldTop) return r;  
    if (bottom == oldTop){  
        bottom = 0,  
        if (top.CAS(oldTop, newTop, oldStamp, newStamp))  
            return r;  
    }  
    top.set(newTop, newStamp); bottom = 0;  
    return null;  
}
```

At most one item left

Take Work

Try to steal last task.

Reset bottom because the

DEQueue will be empty

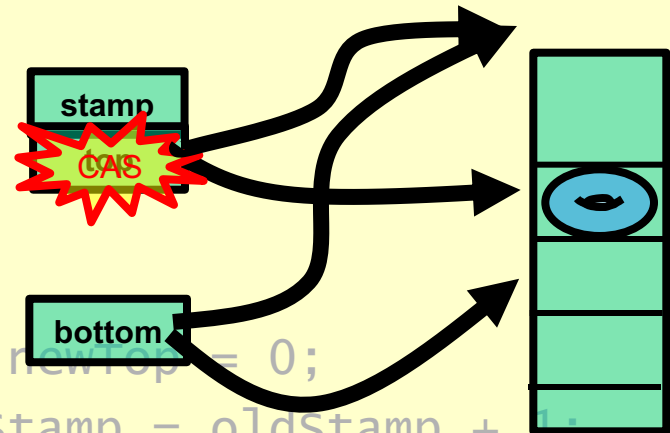
even if unsuccessful (why?)

```
Runnable popBottom() {  
    if (bottom == 0) return null;  
    bottom = top;  
    Runnable r = tasks[bottom];  
    if (tasks[bottom] == null) return null;  
    int oldStamp = stamp[0], newStamp = oldStamp + 1;  
    if (bottom > oldTop) return r;  
    if (bottom == oldTop){  
        bottom = 0;  
        if (top.CAS(oldTop, newTop, oldStamp, newStamp))  
            return r;  
    }  
    top.set(newTop, newStamp); bottom = 0;  
    return null;  
}
```

Take Work

```
Runnable popBottom() {  
    if (bottom == 0) return null;  
    bottom--;  
    Runnable r = tasks[bottom];  
    int[] stamp = new int[1];  
    int oldTop = top.get(stamp), newTop = 0;  
    int oldStamp = stamp[0], newStamp = oldStamp + 1;  
    if (bottom > oldTop) return r;  
    if (bottom == oldTop){  
        bottom = 0;  
        if (top.CAS(oldTop, newTop, oldStamp, newStamp)  
            return r;  
    }  
    top.set(newTop, newStamp); bottom = 0;  
    return null;  
}
```

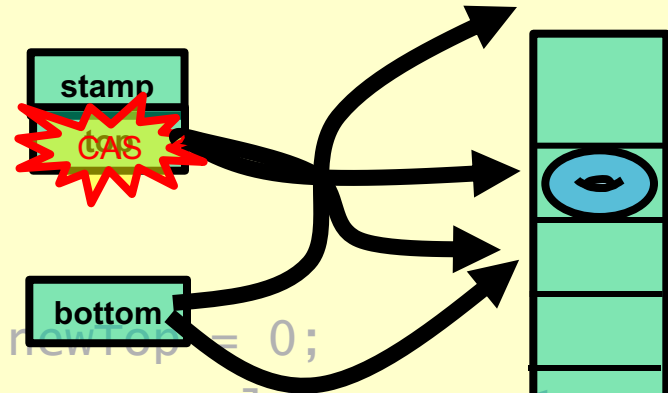
The diagram illustrates the CAS operation in the `popBottom` method. A red starburst labeled "CAS" is positioned between the `stamp` and `bottom` variables. Arrows point from these variables to the `top` stack structure, which is a vertical array of cells. The `top` stack is shown with a blue circle indicating the current top element. The `stamp` variable is associated with the `top` stack, and the `bottom` variable is associated with the `tasks` array.



Take Work

If I lose CAS, thief must have won...

```
Runnable popBottom() {  
    if (bottom == 0) return null;  
    Runnable r = tasks[bottom];  
    int[] stamp = new int[1];  
    int oldTop = top.get(stamp), newTop = 0;  
    int oldStamp = stamp[0], newStamp = oldStamp + 1;  
    if (bottom > oldTop) return r;  
    if (bottom == oldTop){  
        bottom = 0;  
        if (top.CAS(oldTop, newTop, oldStamp, newStamp))  
            return r;  
    }  
    top.set(newTop, newStamp); bottom = 0;  
    return null;  
}
```



Take Work

**Failed to get last task
(bottom could be less than top)**

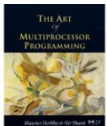
**Must still reset top and bottom
since deque is empty**

```
final TopBottom() {  
    if (bottom == 0) return null;  
    Runnable r = new Runnable() {  
        int[] arr = new int[10];  
        int oldTop = top, newTop = top + 1;  
        int oldStamp = stamp[0], newStamp = oldStamp + 1;  
        if (bottom > oldTop) return r;  
        if (bottom == oldTop){  
            bottom = 0;  
            if (top.CAS(oldTop, newTop, oldStamp, newStamp))  
                return r;  
        }  
    }  
}
```

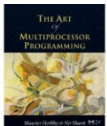
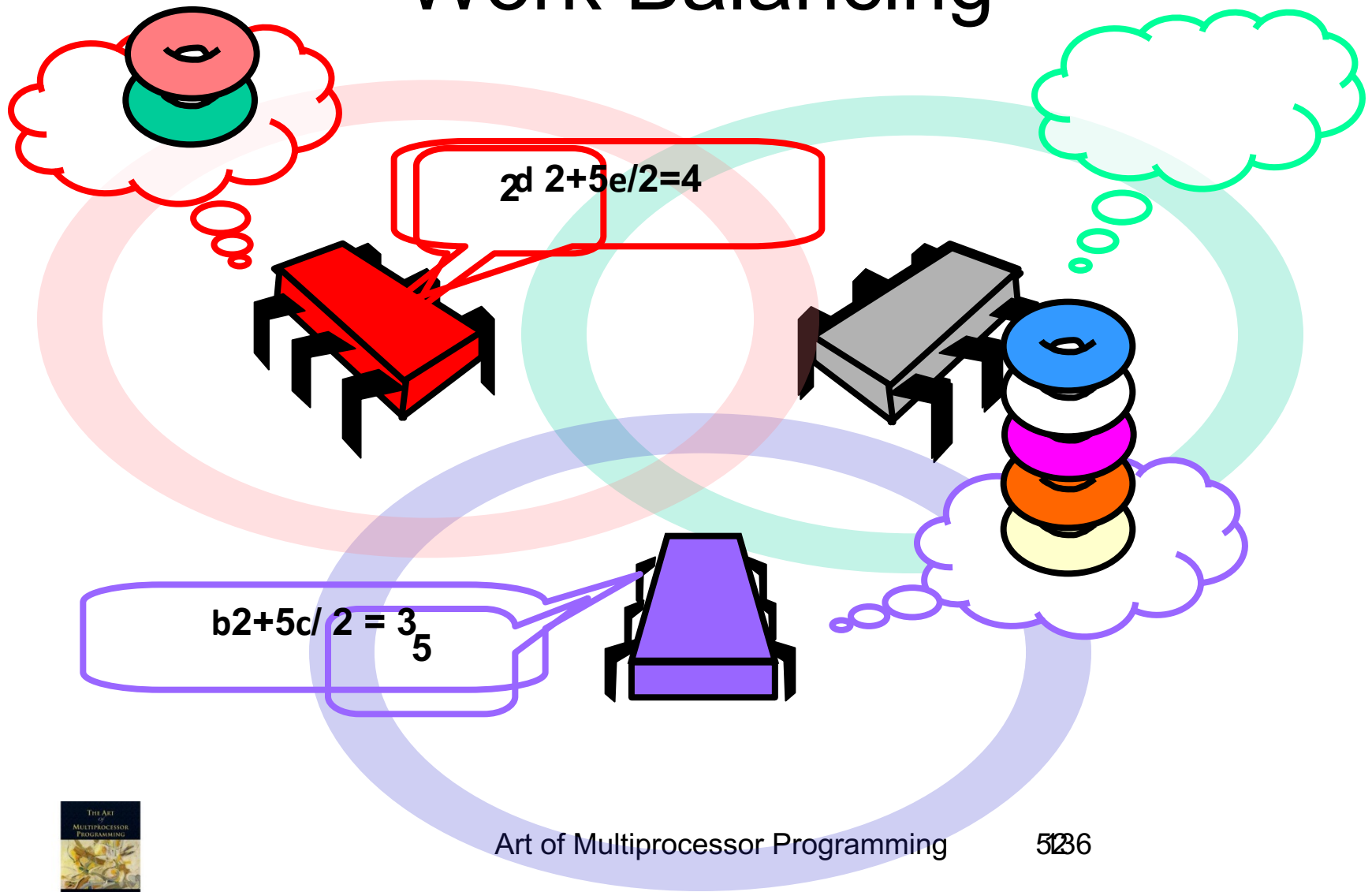
```
top.set(newTop, newStamp); bottom = 0;  
return null;}
```

Variations

- Stealing is expensive
 - Pay CAS
 - Only one task taken
- What if
 - Move more than one task
 - Randomly balance loads?

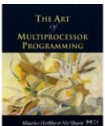


Work Balancing



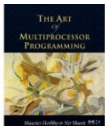
Work-Balancing Thread

```
public void run() {  
    int me = ThreadID.get();  
    while (true) {  
        Runnable task = queue[me].deq();  
        if (task != null) task.run();  
        int size = queue[me].size();  
        if (random.nextInt(size+1) == size) {  
            int victim = random.nextInt(queue.length);  
            int min = ..., max = ...;  
            synchronized (queue[min]) {  
                synchronized (queue[max]) {  
                    balance(queue[min], queue[max]);  
                }  
            }  
        }  
    }  
}
```



Work-Balancing Thread

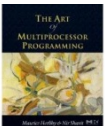
```
public void run() {  
    int me = ThreadID.get();  
    while (true) {  
        Runnable task = queue[me].deq();  
        if (task != null) task.run();  
        int size = queue[me].size();  
        if (random.nextInt(size+1) == size) {  
            int victim = random.nextInt(queue.length);  
            int min = ..., max = ...;  
            synchronized (queue[min]) { Keep running tasks  
                synchronized (queue[max]) {  
                    balance(queue[min], queue[max]);  
                }  
            }  
        }  
    }  
}
```



Work-Balancing Thread

```
public void run() {  
    int me = ThreadID.get();  
    while (true) {  
        Runnable task = queue[me].deq();  
        if (task != null) task.run();  
        int size = queue[me].size();  
        if (random.nextInt(size+1) == size) {  
            int victim = random.nextInt(queue.length);  
            int min = ..., max = ...;  
            synchronized (queue[min]) {  
                synchronized (queue[max]) {  
                    balance(queue[min], queue[max]);  
                }  
            }  
        }  
    }  
}
```

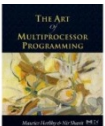
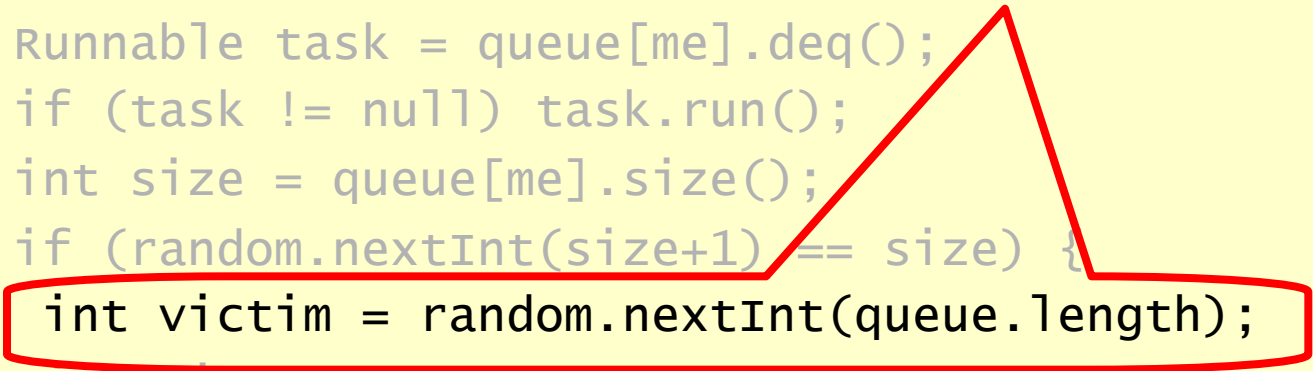
With probability $1/|queue|$



Work-Balancing Thread

```
public void run() {  
    int me = ThreadID.get();  
    while (true) {  
        Runnable task = queue[me].deq();  
        if (task != null) task.run();  
        int size = queue[me].size();  
        if (random.nextInt(size+1) == size) {  
            int victim = random.nextInt(queue.length);  
            int min = ..., max = ...;  
            synchronized (queue[min]) {  
                synchronized (queue[max]) {  
                    balance(queue[min], queue[max]);  
                }  
            }  
        }  
    }  
}
```

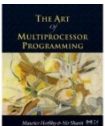
Choose random victim



Work-Balancing Thread

```
public void run() {  
    int me = ThreadID.get();  
    while (true) {  
        Runnable task = queue[me].deq();  
        if (task != null) task.run();  
        int size = queue[me].size();  
        if (random.nextInt(size+1) == size) {  
            int victim = random.nextInt(queue.length);  
            int min = ..., max = ...;  
            synchronized (queue[min]) {  
                synchronized (queue[max]) {  
                    balance(queue[min], queue[max]);  
                }  
            }  
        }  
    }  
}
```

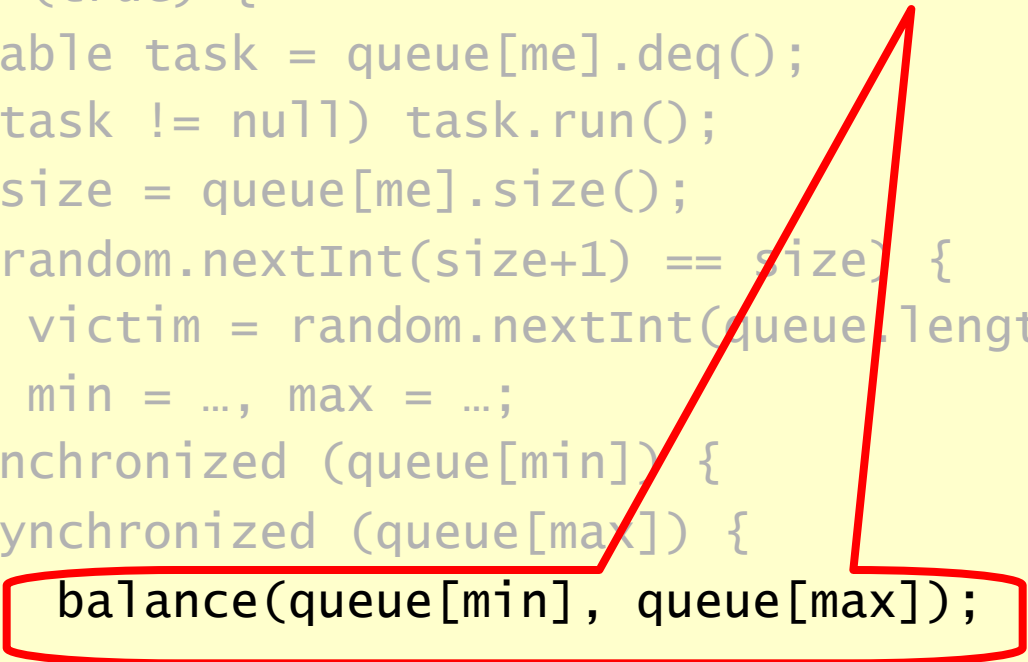
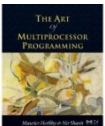
Lock queues in canonical order



Work-Balancing Thread

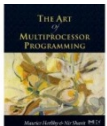
```
public void run() {  
    int me = ThreadID.get();  
    while (true) {  
        Runnable task = queue[me].deq();  
        if (task != null) task.run();  
        int size = queue[me].size();  
        if (random.nextInt(size+1) == size) {  
            int victim = random.nextInt(queue.length);  
            int min = ..., max = ...;  
            synchronized (queue[min]) {  
                synchronized (queue[max]) {  
                    balance(queue[min], queue[max]);  
                }  
            }  
        }  
    }  
}
```

Rebalance queues

A red line originates from the text 'Rebalance queues' and points to the 'balance(queue[min], queue[max]);' line in the code. A red box encloses this line, and the line also extends to the closing braces of the nested synchronized blocks.

Work Stealing & Balancing

- Clean separation between app & scheduling layer
- Works well when number of processors fluctuates.
- Works on “black-box” operating systems



How to write parallel programs?

- Split a program into parallel parts
- In an effective way
- Requires managing threads and work packages
- Some approaches:
 - BSP
 - Fan-out-fan-in
 - Pipelines
 - Map-reduce
 - Producer-Consumer
 - Fork-join